



Universidad
Politécnica
de Madrid

**Programación
Hardware
Reconfigurable**



Escuela Técnica Superior
de Ingeniería de
Sistemas Informáticos

PROYECTO FINAL DE LA ASIGNATURA

FBP

(FPGA Based Plotter)

Nombre del equipo: PHR26-M03

Miembros del equipo:

Líder	Alumno
X	Ferreira Souza, Jhonatan
	Blanco Tello, Víctor
	Navarro Muñoz, Alberto
	Oganesyan Avagyan, Tigrán

ÍNDICE

ÍNDICE.....	1
MEMORIA DESCRIPTIVA.....	2
1. OBJETO Y DESCRIPCIÓN DEL PROYECTO.....	2
1.1. OBJETO DEL PROYECTO.....	2
1.2. DESCRIPCIÓN DEL PROYECTO.....	2
2. PROCESADO EXTERNO.....	3
2.1. DATOS DE ORIGEN.....	3
2.2. G-CODE Y EMPAQUETADO BINARIO.....	3
2.3. CÁLCULOS DE CINEMÁTICA.....	4
2.4. ENVÍO DE PAQUETES.....	6
3. PROCESADO INTERNO.....	7
3.1. SUB-MÓDULOS.....	7
3.1.1. RECEPTOR DE INSTRUCCIONES.....	7
3.1.2. TRANSMISOR DE CONFIRMACIONES.....	7
3.1.3. ENSAMBLADOR DE PAQUETES.....	7
3.1.4. ALGORITMO DE BRESENHAM.....	8
3.2. MÓDULO PRINCIPAL.....	9
4. CIRCUITO DEL SISTEMA.....	9
5. CONSIDERACIONES FUTURAS.....	10
6. CONCLUSIÓN.....	10
BIBLIOGRAFÍA.....	12

MEMORIA DESCRIPTIVA

1. OBJETO Y DESCRIPCIÓN DEL PROYECTO

1.1. OBJETO DEL PROYECTO

El presente proyecto consiste en la implementación de un sistema de control numérico por computador (CNC) [1], de tipo trazador, utilizando una FPGA como controlador principal del circuito de motores a paso.

El objeto de este proyecto es el diseño y justificación de los pasos realizados para una primera iteración básica de un sistema basado en el principio coreXY [2], sistema cinemático de movimiento encontrado en impresoras 3D y máquinas CNC.

1.2. DESCRIPCIÓN DEL PROYECTO

El presente proyecto recoge la descripción de la implementación de un conjunto de máquinas de estado finito (MEF) en una FPGA modelo Basys Artix-7 para el funcionamiento del sistema, así como los componentes físicos necesarios para la construcción e implementación de un trazador portable.

Los componentes físicos para la construcción del modelo consisten en:

- 2 x Extrusiones de aluminio 20 x 40 de 560 mm
- 2 x Extrusiones de aluminio 20 x 40 de 480 mm
- 2 x Extrusiones de aluminio 20 x 40 de 520 mm
- 1 x Rail Lineal de 400 mm
- 2 x Correas dentadas GT2 de 6 mm de ancho y 2 m de largo
- 10 x Poleas sin dentar, Z20 de 5 mm de diámetro.

Los componentes electrónicos necesarios para el sistema consisten en:

- 1 x Placa de Desarrollo FPGA Basys 3
- 2 x Motores a paso basados en NEMA 17 | BJ42D15-26V12
- 1 x Servomotor SG90
- 2 x Controladores A4988
- 2 x Condensadores electrolíticos de 100 μ F
- 1 x Fuente de alimentación CC 12V
- 1 x Fuente de alimentación CC 5V
- 1 x Módulo Convertidor reductor LM2596

Así mismo, el proyecto queda dividido en dos principales zonas de procesado: el procesado externo, orientado al preprocesado de datos; y el procesado interno, orientado a ensamblado de instrucciones y envío de señales a los motores.

2. PROCESADO EXTERNO

2.1. DATOS DE ORIGEN

El objetivo principal de FBP es poder realizar un dibujo físico a partir de un fichero de imagen o dibujo cualquiera. Utilizando la herramienta “Image2Gcode” [3], se puede partir de cualquier archivo de imagen de formato convencional (JPEG, PNG, BMP, etc), así como ficheros de gráficos de vectores escalables (SVG), dónde se describe una imagen vectorialmente en texto [4].

Como indica el nombre de la herramienta, el objetivo es convertir una imagen en un dato procesable, que en éste caso se trata del código G, estándar utilizado en CNC e impresión 3D. La ventaja principal que presenta el uso de una herramienta como la anterior es que al trabajar con amplia compatibilidad, los comandos deben ser sencillos, y el contorno del dibujo debe ser detectado mediante un algoritmo de visión artificial, siendo en este caso Canny [5].

2.2. G-CODE Y EMPAQUETADO BINARIO

El código G [6], mejor conocido por el anglicismo G-code, es un lenguaje mediante un usuario puede comunicar a una máquina herramienta qué hacer y cómo debe hacerlo.

G-code posee una estructura muy sencilla para indicar una instrucción, donde se utiliza una letra de código, la cual indica el tipo de comando, y un número de parámetro, utilizado para distinguir diferentes comandos. Anteriormente, se mencionó la sencillez que debe tener un conversor de imagen a G-code, y en este caso trabajaremos con las instrucciones ‘G0’ y ‘G1’, utilizadas para movimientos lineales [7].

Aun así, G-code sigue siendo un formato complejo para procesar en una FPGA, debido a que un dibujo se define mediante varias líneas de G-code, las cuales son cadenas de bytes demasiado largas para hacer un procesamiento eficiente. La solución que se ha adoptado es una codificación línea a línea de un fichero de instrucciones G-code en diferentes paquetes binarios de tamaño fijo y reducido, simplificando el dato a manejar. Para ello, se ha decidido enviar la información en paquetes de 6 bytes, donde se puede ver la información estructurada en la siguiente figura:

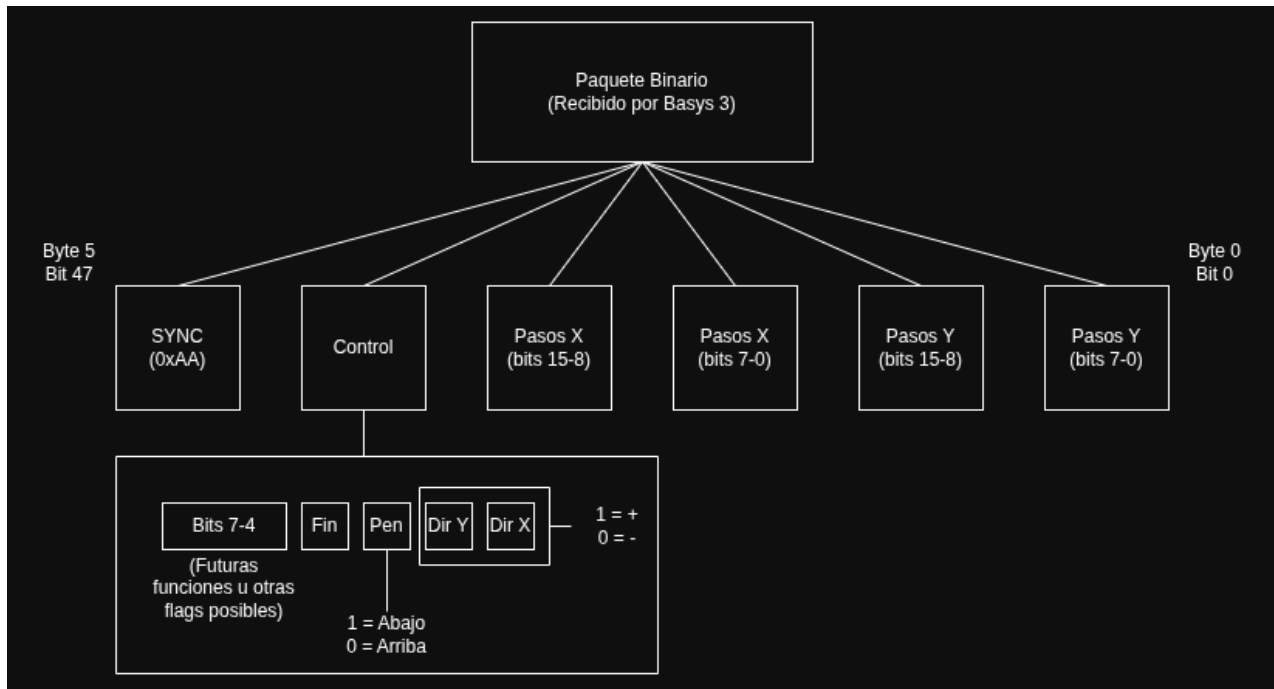


Figura 1: Paquete binario de instrucción G-code (fuente propia)

El primer byte del paquete se trata de la trama de sincronización, la cual servirá como bandera para indicar el inicio de un comando. El siguiente byte es el de control, del cual sólo usamos los 4 bits de menor peso. Cada bit representa la dirección a moverse en cada eje, según el signo de la coordenada; el estado del lápiz y una bandera de finalización del dibujo. Los siguientes bytes indican el número de pasos a moverse en cada eje, conversión realizada en la parte externa, utilizando el siguiente factor de conversión:

$$\text{Distancia en steps} = \text{Distancia mm} * \text{Steps por mm}$$

$$\text{Steps por mm} = \frac{\text{Pasos por rotación} * \mu\text{Steps controlador}}{\text{Paso correa (mm)} * \text{dientes polea}}$$

Gracias a esta codificación, nos será más sencillo desempaquetar la información en la MEF encargada de traducir estos bytes a señales de paso para los motores.

2.3. CÁLCULOS DE CINEMÁTICA

Antes de poder realizar cualquier dibujo, se debe tener en cuenta los siguientes factores físicos:

Los motores NEMA 17 cuentan con un ángulo de 1,8° de rotación por paso, por lo que una revolución entera cuenta con 200 pasos [8]. Las correas GT2 tienen un paso de 2 mm, y nuestras poleas son de 20 dientes, dejándonos con la cifra 80 pasos por mm de dibujo. Se ha de notar también, que al estar trabajando con un área de dibujo limitada, en este caso hojas de tamaño A4, en el empaquetado se tiene en cuenta un factor de escala para normalizar las coordenadas al área de dibujo útil. Además, la FPGA recibirá posiciones relativas respecto a los movimientos anteriores, haciendo el sistema más portable y compatible con distintos tamaños de dibujo.

Originalmente, en una primera iteración, FBP dibujaba a una velocidad fija y constante. En principio esto no suena a que presente ningún problema, sin embargo, al trabajar a grandes velocidades, y con el añadido que un dibujo puede presentar una secuencia rápida de vértices muy juntos, puede aparecer lo que se conoce como efecto ‘ghosting’ o ‘ringing’ [9]. Esto visualmente presentaría patinazos o líneas imprecisas en nuestro dibujo.

Para evitarlo, se ha decidido implementar artificialmente un perfil de velocidad trapezoidal en la FPGA. El objetivo es conseguir que los motores se muevan a una velocidad y aceleración como se muestra en la siguiente figura:

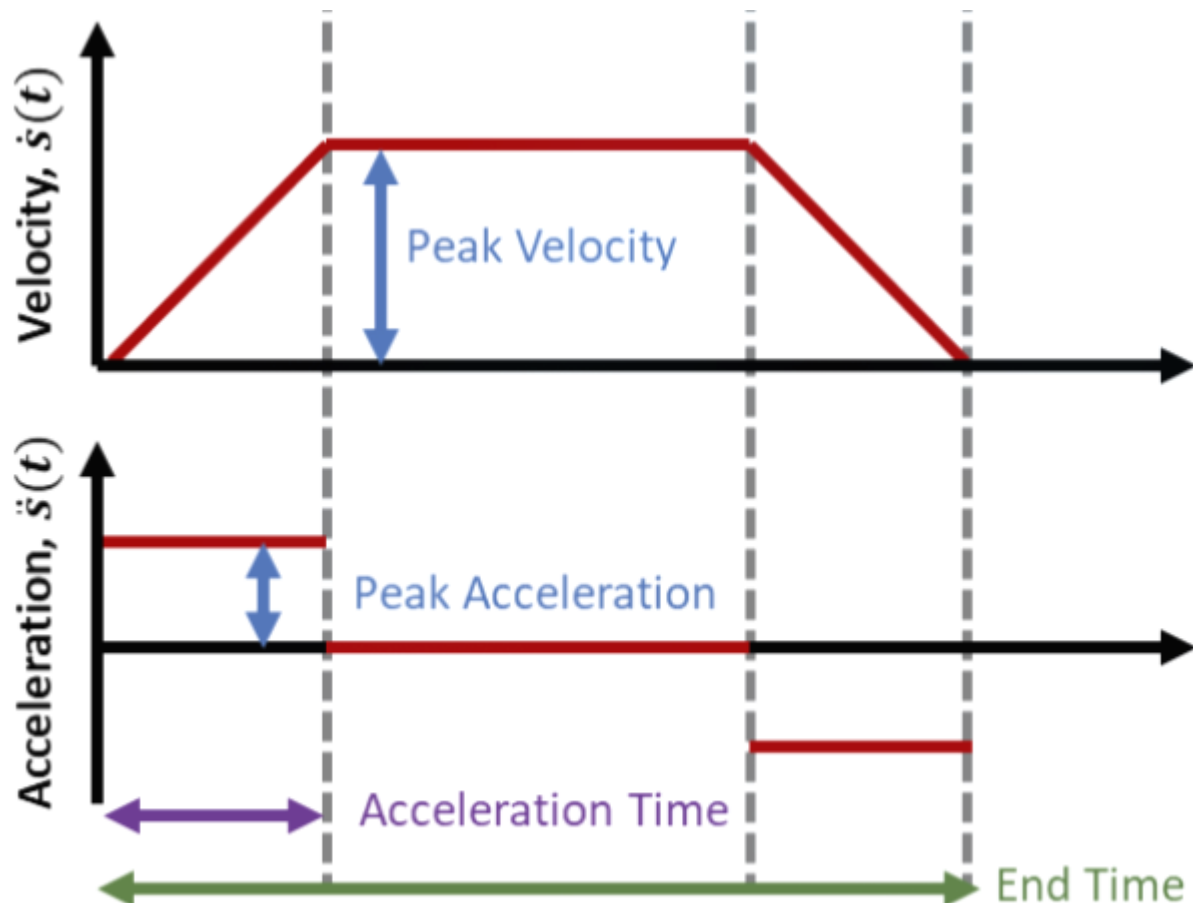


Figura 2: Perfiles de velocidad y aceleración trapezoidales [10]

Al estar trabajando con pulsos en vez de velocidad o distancia, se debe de encontrar un modo de representar las rampas de aceleración y de frenado. En un cronograma de señales de pulso, esto representa la distancia que hay entre pulso y pulso, siendo esta inversamente proporcional a la velocidad del motor. En la siguiente figura se puede ver claramente dicha representación:

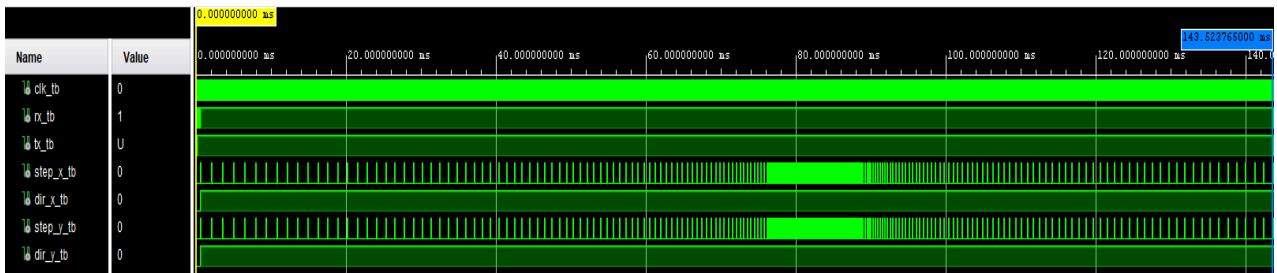


Figura 3: Perfil de velocidad trapezoidal representado en pulsos de paso (Fuente propia)

Como se puede apreciar en la figura, los pulsos van reduciendo su distancia a medida que aceleran, debido a que hay menos retardo entre pulso y pulso. Ésto se realiza hasta llegar a una velocidad de crucero, siendo ésta la velocidad máxima de dibujo. Una vez se recorre la distancia hasta la rampa de frenado, la FPGA añadirá de nuevo un retardo creciente entre pulso y pulso, hasta llegar a la velocidad mínima de funcionamiento.

En un sistema que implemente un cálculo de perfil trapezoidal correctamente, se debe recurrir a cálculos físicos utilizando derivación y racionalización. Es totalmente posible implementar dichas operaciones en la FPGA, así como procesar dichos cálculos de antemano y enviarlo a la placa como información extra, pero debido al enfoque del proyecto, se ha decidido calcular artificialmente dicha rampa de aceleración, con una distancia fija en pasos.

2.4. ENVÍO DE PAQUETES

Una vez hechos todos los cálculos anteriores, se debe empaquetar cada byte en un paquete que será enviado mediante protocolo UART a la placa por el puerto micro-USB de esta misma. Según el manual de referencia de la Basys 3, el propio puerto cuenta con un chip de puente que permitirá comunicar el script de Python con la placa utilizando un puerto serial. Se puede observar el esquema de dicho protocolo de comunicación en la siguiente figura:

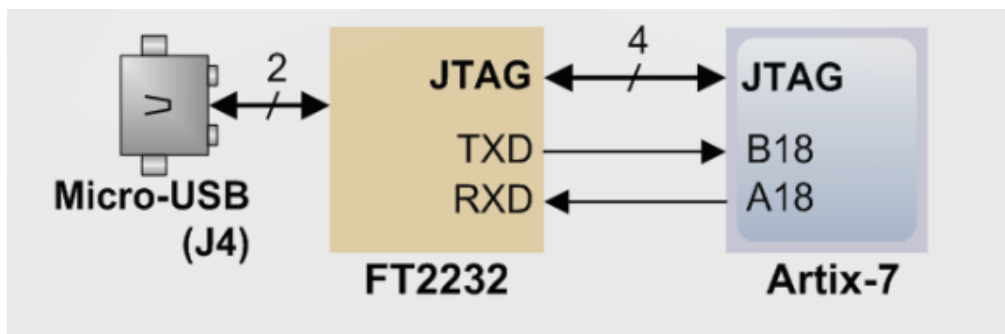


Figura 4: Puente UART Basys 3 [11]

Dicho puente también nos permitirá enviar datos desde la FPGA a la parte externa, permitiendo implementar un protocolo de sincronización que posteriormente se explicará.

3. PROCESADO INTERNO

3.1. SUB-MÓDULOS

3.1.1. RECEPTOR DE INSTRUCCIONES

El primer módulo de la implementación es el módulo receptor de UART. El principal problema es la velocidad de funcionamiento, ya que al tratarse de comunicación serial, ésta es mucho más lenta que el reloj interno de la FPGA. Ésto puede resultar en la pérdida de datos si no se lee justamente en el tiempo preciso. Para ello, deberemos calcular cuánto dura un bit cuando éste llega al pin de recepción. Para una conexión serial a 115200 baudios:

$$Duración = \frac{Freq.Clk\ FPGA}{Freq.UART} = \frac{100MHz}{115200} = 868\text{ ciclos}$$

Conociendo ésto, se puede implementar un contador hasta la mitad de la duración del bit, para leer exactamente en el centro del pulso (y así evitar ruido de la señal) y posteriormente hacer saltos de 868 ciclos hasta leer 8 bits, donde luego mediante una señal interna, se avisará de la correcta lectura de 1 bit.

3.1.2. TRANSMISOR DE CONFIRMACIONES

Otro módulo necesario para que el sistema funcione correctamente es el módulo de transmisión de UART. En principio no parece necesario, pero al tener en cuenta los tiempos de ejecución de cada paquete, se necesita algún tipo de control de flujo de recepción de paquetes. Sin un mecanismo de sincronización, la FPGA puede recibir datos mientras aún se están enviando pulsos a los motores, pudiendo resultar en una sobreescritura de los datos y un potencial fallo grave en el dibujo. Para solucionarlo, la FPGA necesitará comunicarse también al exterior.

Con una MEF similar a la del módulo de recepción, se implementa un contador hasta el número de ciclos por bit a transmitir en UART, y se avisará a una bandera interna la finalización de la transmisión cuando se detecte que se ha enviado un byte entero.

Se debe tener en cuenta, que la velocidad limitante del procesamiento de paquetes es la de la comunicación serial, además de que si se está procesando un comando largo, pueden haber esperas mayores entre línea y línea.

3.1.3. ENSAMBLADOR DE PAQUETES

Una vez recibida la información, la FPGA necesita un mecanismo para segmentar y almacenar correctamente los bytes recibidos. Para ello se implementará una MEF, con un número de estados igual al número de bytes del paquete. En cada estado, se almacenará la información recibida por el módulo de recepción UART, en su correspondiente registro, los cuales serán accedidos posteriormente por aquellos módulos que necesiten dicha información. Para evitar esperar un segundo ciclo para leer el registro del último byte, se ha decidido también aprovechar la concurrencia para concatenar directamente los últimos bits recibidos al último registro, siendo éste los bits bajos del eje Y; consiguiendo ahorrar un registro.

3.1.4. ALGORITMO DE BRESENHAM

Las coordenadas de un fichero G-code suelen ser representadas por un número con coma flotante. Este tipo de dato suele tener bastantes complicaciones a la hora de realizar operaciones con ellos, debido a la pérdida de precisión y la ineficiencia en operaciones como la multiplicación o la división. Además, como se comentó anteriormente, el dato que lee la FPGA es un número entero de pasos, siendo éstos el número de pulsos que mandará el controlador a los motores. Esto también conlleva una importante pérdida de información, ya que en algún punto del dibujo, un motor puede llegar a no moverse si necesita hacer un movimiento que tenga una medida menor a la distancia que recorre un paso, como en líneas diagonales o curvas.

En la siguiente figura, podemos ver el problema visualmente, donde se observa la recta ideal en rojo, los errores posibles delimitados por las rectas verdes, y los cuadros amarillos siendo una posible solución del rasterizado de la línea:

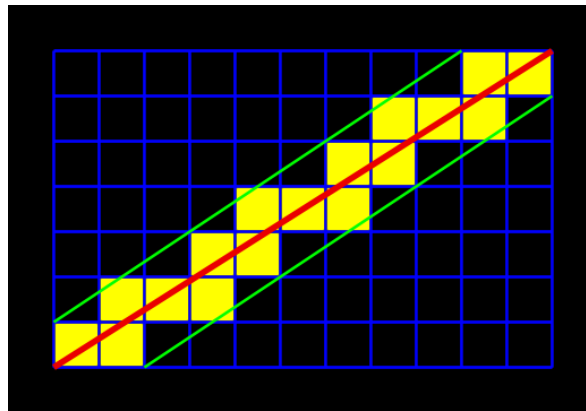


Figura 3: Visualización del algoritmo de Bresenham [12]

Para ello, se ha decidido implementar el algoritmo de Bresenham, un método para rasterizar aquellas líneas que no son rectas perfectas utilizando únicamente operaciones con enteros [13]. En el ámbito del proyecto, debido a que se trabaja con únicamente dos ejes de dirección, la acumulación de error irá asignada por un mecanismo de maestro-esclavo, donde el motor que más pasos tenga que dar se denominará maestro, y el que menos tenga que dar se denominará esclavo. Esto resulta en otra MEF, en la que se enviarán pulsos de acorde al número de pasos del eje maestro. Cada vez que el motor maestro dé un paso, se suma a un acumulador la distancia total del eje esclavo. Si el acumulador supera el límite, significa que el eje menor debe dar un paso, siendo el límite la distancia del eje maestro. Al estar trabajando con concurrencia, en ese mismo ciclo de reloj también restamos el límite para reiniciar el acumulador.

Dicho algoritmo soluciona completamente el problema anterior, debido a que todo el dibujo se encuentra definido únicamente por movimientos lineales, y al momento de realizar movimientos como curvas o diagonales, se tiene en cuenta el error acumulado.

3.2. MÓDULO PRINCIPAL

Con todos los sub-módulos anteriores, la FPGA ya puede comenzar a funcionar como un controlador CNC. Para ello, se conectarán instancias de dichos módulos en una capa superior. En éste módulo principal se definirán las entradas y las salidas de nuestro sistema, así como la MEF que se encargará de sincronizar la FPGA con la parte externa a través de un mecanismo similar a un handshake de paquetes ‘ACK’ en redes de computadores.

Dicho mecanismo se encargará de mandar un paquete de dos bytes, con la palabra ‘OK’, tras terminar de enviar el último pulso dirigido a los motores. En la parte externa se puede incluso realizar una comprobación del correcto envío de dicho paquete, reforzando el sincronismo de ambos sistemas. La MEF del mecanismo ping-pong anteriormente descrito, puede verse en la siguiente figura:

4. CIRCUITO DEL SISTEMA

En cuanto a electrónica, se puede ver el circuito en el siguiente diagrama:

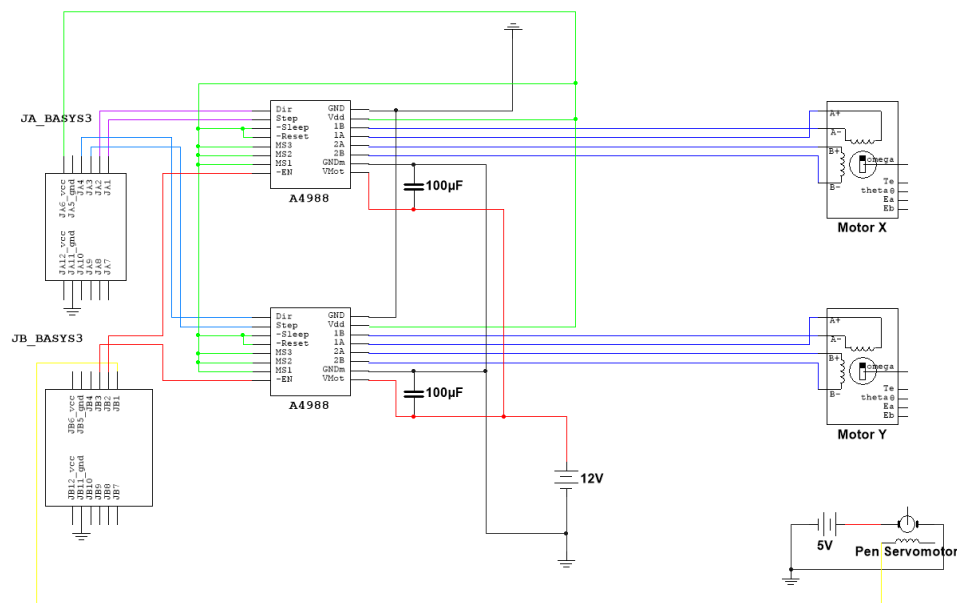


Figura 4: Diagrama del circuito electrónico FBP (fuente propia)

El control de motores está asignado al conector Pmod JA, donde se sacan las señales de dirección y paso hacia los controladores. Además, la lógica de éstos se alimenta por un pin VCC de dicho conector. El control del servomotor del cabezal está asignado al conector Pmod JB, desde el cual también saldrán las señales Enable de los controladores. Esta separación se ha hecho precisamente para evitar ruido electromagnético entre las señales de los motores.

Es de notar que la masa común del circuito es la de la propia FPGA, y que la fuente de alimentación de 5V del servomotor se proporcionará a través de un convertidor reductor de 12V a 5V. En cuanto a

corriente, los motores están especificados a 0.84A por fase, generando un consumo de corriente nominal de unos 3 a 4 amperios. A esto le sumamos unos 0,5A aproximados de consumo del servomotor, teniendo un consumo de corriente nominal de unos 4,5-5A. Dicho consumo será satisfecho por una fuente de alimentación CC de 12V de 90W, que puede funcionar nominalmente hasta los 6A.

5. CONSIDERACIONES FUTURAS

Durante el desarrollo del presente proyecto se han ido encontrando varias funcionalidades que se han decidido no implementar o implementar en futuras iteraciones debido a que quedaban fuera del ámbito de la asignatura y del proyecto en sí.

Una de ellas es el calibrado automático del punto de origen. Actualmente, los motores deben ser movidos manualmente al punto de origen del área de dibujo. En caso de querer implementar un sistema de 'homing' similar a aquellos encontrados en máquinas CNC o impresoras 3D, se deberían incluir finales de carrera en las extrusiones del FBP, los cuales enviarán una señal a la FPGA y sería transmitido por el módulo de transmisión un paquete de 1 byte; en el que se indicaría a la parte externa que se ha tocado. Además, la FPGA necesitaría estar constantemente transmitiendo su posición actual absoluta al software externo, para así poder determinar correctamente el área máxima de dibujo y el punto de origen.

También se ha considerado implementar una pantalla de información en tiempo real, o por lo menos enviar una señal de vídeo con los datos correspondientes. Esto sería bastante útil a la hora de comprobar el funcionamiento de los motores, y sería cómodo al momento de necesitar comprobar velocidades.

En caso de querer hacer un producto final, sin depender de la Basys 3, se podría escribir el código de procesado en un microcontrolador, o cualquier circuito integrado programable para que actuase como controlador interno, y así poder hacer el FBP completamente portable. Además, se podrían incluir módulos de lectura de datos similar a lo implementado en impresoras 3D, teniendo soporte para USB y otros formatos como tarjetas microSD.

6. CONCLUSIÓN

Siguiendo la ejecución de los pasos anteriormente descritos, y considerando las limitaciones físicas del circuito, es posible implementar un sistema CNC trazador utilizando una FPGA Basys 3 Artix 7 como controlador principal. En dicha placa, irán implementadas las máquinas de estado finito necesarias para poder traducir instrucciones de código G a un formato legible por la FPGA, que será procesado internamente y generará las señales necesarias hacia los motores para poder rasterizar una imagen en un dibujo físico.

Página intencionalmente dejada en blanco.

BIBLIOGRAFÍA

- [1] «Control numérico», Wikipedia, la enciclopedia libre. 4 de enero de 2026. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Control_num%C3%A9rico&oldid=171336169
- [2] «CoreXY | Cartesian Motion Platform». Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://corexy.com/theory.html>
- [3] TheNano, LittleSurvival/image2gcode. (19 de marzo de 2026). Python. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://github.com/LittleSurvival/image2gcode>
- [4] «Gráficos vectoriales escalables», Wikipedia, la enciclopedia libre. 15 de enero de 2026. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Gr%C3%A1ficos_vectoriales_escalables&oldid=171520403
- [5] «Algoritmo de Canny», Wikipedia, la enciclopedia libre. 16 de febrero de 2024. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Algoritmo_de_Canny&oldid=158232089
- [6] «G-code», Wikipedia, la enciclopedia libre. 30 de abril de 2024. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://es.wikipedia.org/w/index.php?title=G-code&oldid=159813147>
- [7] thinkyhead, «Linear Move», Marlin Firmware. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://marlinfw.org/docs/gcode/G000-G001.html>
- [8] 262588213843476, «Creality Ender 3 Stock Factory Vref», Gist. Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://gist.github.com/knoopx/e6c40a009e796203b93a75a3ed6a5ab8>
- [9] «Ghosting (Echo/Ringing Effect) | Polymaker Wiki». Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://wiki.polymaker.com/printing-tips/common-printing-issues/ghosting-echo-ringing-effect>
- [10] «Design Trajectory with Velocity Limits Using Trapezoidal Velocity Profile». [En línea]. Disponible en: <https://www.mathworks.com/help/robotics/ug/design-a-trajectory-with-velocity-limits-using-a-trapezoidal-velocity-profile.html>

[11] «Basys 3 Reference Manual | USB-UART Bridge». [En línea]. Disponible en:
https://digilent.com/reference/programmable-logic/basys-3/reference-manual#usb-uart_bridge_serial_port

[12] [13] «Algoritmo de Bresenham», Wikipedia, la enciclopedia libre. 13 de abril de 2025. Accedido: 11 de mayo de 2026. [En línea]. Disponible en:
https://es.wikipedia.org/w/index.php?title=Algoritmo_de_Bresenham&oldid=166823838